

Real-Time Scheduling — Beginner-Friendly Notes

Generalizations of RMA, Precedence, and Aperiodic Servers

Lecture 8 & 9 — Made simple

How to read this document

Each section starts with the "big picture" idea in plain language, then gives an example, then a diagram. Don't worry about all the math symbols on the first pass — focus on understanding WHY each idea exists. The vocabulary at the very end will help if get lost.

Before Start: Mini-Vocabulary

A real-time task is just a piece of code (a job) that must finish before some deadline. Three properties matter most:

- **Execution time (e or C):** how long the task runs on the CPU when nothing interrupts it.
- **Period (p or T):** for periodic tasks, how often the task arrives (e.g., every 10 ms).
- **Deadline (d or D):** how long after the task arrives it must be finished.

In simple RMA, deadline = period. The whole point of today's lecture is to look at all the cases where this is no longer true, and other "real-world" complications.

Big Picture: Real-Time Scheduling Approaches

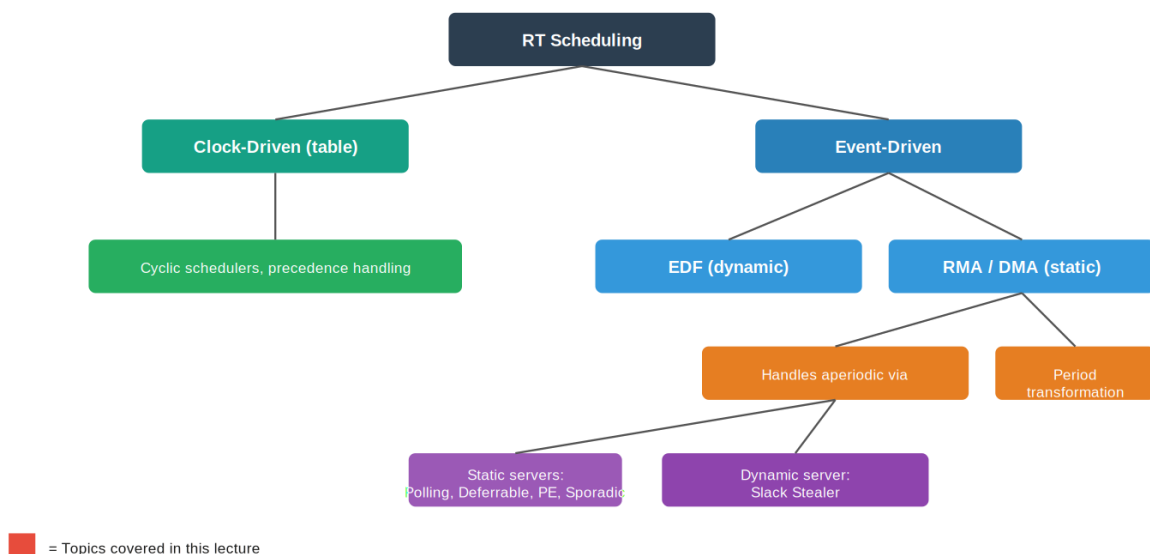


Figure 0. Where today's topics fit in the big picture of real-time scheduling.

Part 1: When Deadline $\neq$ Period (DMA)

Plain English: Rate Monotonic Algorithm (RMA) was designed assuming a task's deadline equals its period. But what if a task must finish much earlier than its next arrival? For example, a sensor reading might arrive every 100 ms but need a result back within 20 ms.

The fix: Deadline Monotonic Algorithm (DMA)

- RMA assigns priority based on **period** (shorter period $\rightarrow$ higher priority).
- DMA assigns priority based on **deadline** (shorter deadline $\rightarrow$ higher priority).

Key insight

DMA is the most "optimal" static algorithm when deadlines can be shorter than periods. If DMA fails to schedule a set of tasks, RMA will also fail. When deadline = period, they produce the same schedule.

Part 2: Context Switch Overhead

Plain English: When the CPU switches from running Task A to Task B, the operating system needs a tiny bit of time to save A's state and load B's state. This is called a context switch (CS). In real life, this is never free.

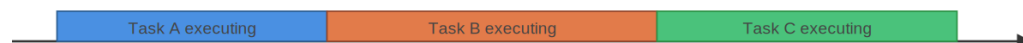
How much overhead per task?

- In the **worst case**, every task has at most 2 context switches:
- 1) When it **starts** (preempts another task)
- 2) When it **finishes** (gives CPU back)

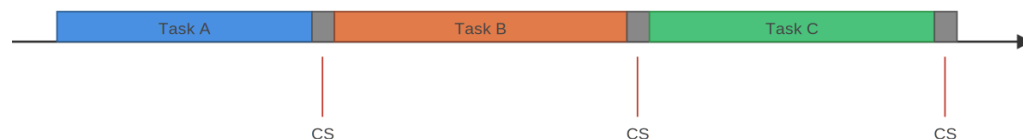
So instead of using e_i as the execution time, use $e_i + 2C$ where C is the time of one context switch.

Context Switch Overhead in Real-Time Tasks

Without overhead:



With context switch overhead (C ms each switch):



Each task incurs at most 2 context switches: when it starts and when it ends. So effective execution time = $e_i + 2C$

Figure 1. The grey blocks are the small gaps the OS uses to switch tasks.

Worked example

Tasks (e, p): $T1=(2, 10)$, $T2=(5, 25)$, $T3=(10, 50)$, and $C = 1$ ms.

Add $2C = 2$ to each execution time:

T1: $e=2+2=4$, $p=10$ T2: $e=5+2=7$, $p=25$ T3: $e=10+2=12$, $p=50$

Now apply Liu and Layland's utilization test or Lehoczky's test as usual. If it still passes, the task set is schedulable even after accounting for context switches.

Part 3: Long Periods → Period Transformation

Plain English: In RMA, long-period tasks get low priority. If such a task is also CRITICAL (must not miss its deadline even during overload), this is dangerous. The trick is to "pretend" it is many smaller tasks with shorter periods, so it gets higher priority.

The trick (Sha & Rajkumar)

Take a critical task with execution time e and period p , and virtually split it into k subtasks:

- Each subtask: execution time = e / k , period = p / k

The task itself is NOT physically changed. We just trick the scheduler into thinking it's many small tasks so it gets higher priority.

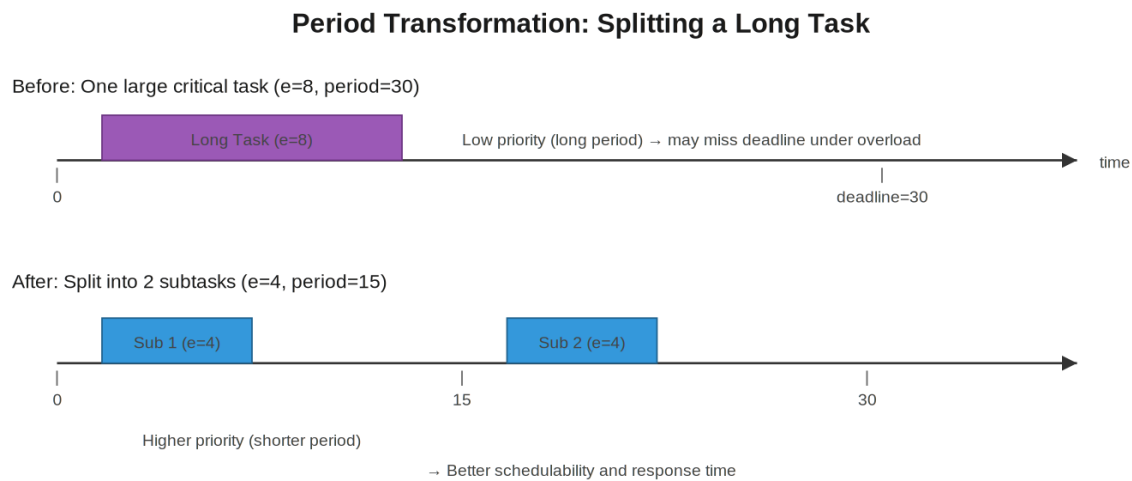


Figure 2. The critical task is virtually split into two halves with a shorter period each.

When this helps

Useful when a critical task would otherwise be starved by transient overload. Trade-off: more scheduling overhead, and have to make sure the task is logically splittable into chunks.

Part 4: Tasks with Dependencies (Precedence Constraints)

Plain English: So far we assumed tasks are independent. But often "Task B depends on Task A finishing first." (Like: filter the signal first, THEN run object detection.) Pure RMA/EDF doesn't handle this naturally — we have to add logic.

Algorithm: Table-driven approach

1. Sort tasks by deadline, ascending (T1 has earliest deadline, Tn has latest).
2. **Start from the right end** (the task with the latest deadline).

- 3. Schedule each task **as late as possible**. Move backwards, always picking tasks whose successors are already scheduled.
- 4. Once a feasible schedule exists, **shift everything left** (as early as possible without violating arrival times). This minimises response time.

Scheduling with Precedence Constraints

Step 1: Tasks with dependencies



Step 2: Schedule from rightmost end (latest deadline first), place each task as late as possible



Step 3: Shift all tasks as early as possible (minimize response time)



Feasible schedule with minimized response time, no violation of dependencies

Figure 3. The three-step procedure: schedule from right (latest deadline) → compact to the left.

Alternative: Retrofit existing schedulers

A "hack" approach: modify each task's arrival time and deadline so that a successor can't possibly start before its predecessor finishes. Then EDF or RMA will work without knowing about the dependencies. It's clever but tricky to get right in practice.

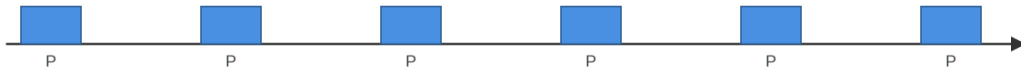
Part 5: Aperiodic and Sporadic Tasks

Plain English: Real systems don't just have nicely periodic tasks. They also have:

- **Aperiodic tasks:** arrive at unpredictable times, no minimum gap (e.g., user button presses).
- **Sporadic tasks:** also unpredictable, BUT we promise a minimum gap between arrivals (e.g., emergency-stop interrupt).

Three Types of Real-Time Tasks

1) Periodic — arrives at regular fixed intervals (e.g., sensor reading every 10ms)



2) Aperiodic — arrives at random times, no minimum gap guarantee (e.g., user clicks)



3) Sporadic — random arrivals BUT with a guaranteed minimum gap (e.g., emergency stop button)



Aperiodic + sporadic are tricky for RMA because RMA assumes periodic behavior

Figure 4. The three task types side-by-side.

The dilemma

If we give aperiodic tasks high priority, they can flood the CPU and break RMA's assumptions. If we give them low priority, critical ones will miss deadlines. Solution: an "Aperiodic Server" — a special periodic task whose job is to serve aperiodic ones in a controlled way.

Part 6: Aperiodic Servers

Idea: invent a "fake" high-priority periodic task called a server. Every period, the server has a budget of CPU time it can spend handling aperiodic requests. There are different policies for what to do when the server is invoked but no aperiodic task is waiting.

6.1 Polling Server (the simplest)

Behaviour: At each server period, IF there is an aperiodic task waiting, serve it (up to the budget). IF NOT, give up the slot and wait for next period.

Polling Server (Static, $e_s=2$, $p_s=5$)

If no aperiodic task at the period boundary, the server gives up its slot and waits for next period

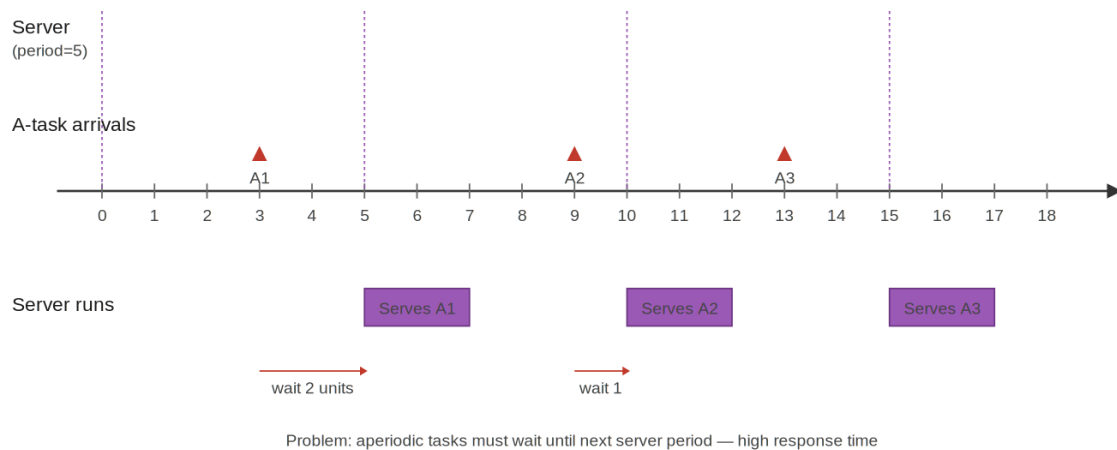


Figure 5. Polling server: aperiodic tasks A1, A2, A3 must wait for the next server period boundary.

Pros and cons

Pro: simplest to implement, periodic schedulability is well preserved. Con: worst response time for aperiodic tasks because they may have to wait an entire period before being served.

Schedulability check

Treat the server as just another periodic task. Add its utilization (e_s / p_s) to the total and check Liu-Layland:

$$\sum (e_i / p_i) + e_s / p_s \leq N * (2^{(1/N)} - 1)$$

6.2 Deferrable Server

Behaviour: Like polling, but if no aperiodic task is waiting at the period start, the server DOESN'T give up its budget. It keeps it (defers it) until an aperiodic task actually arrives, then runs immediately.

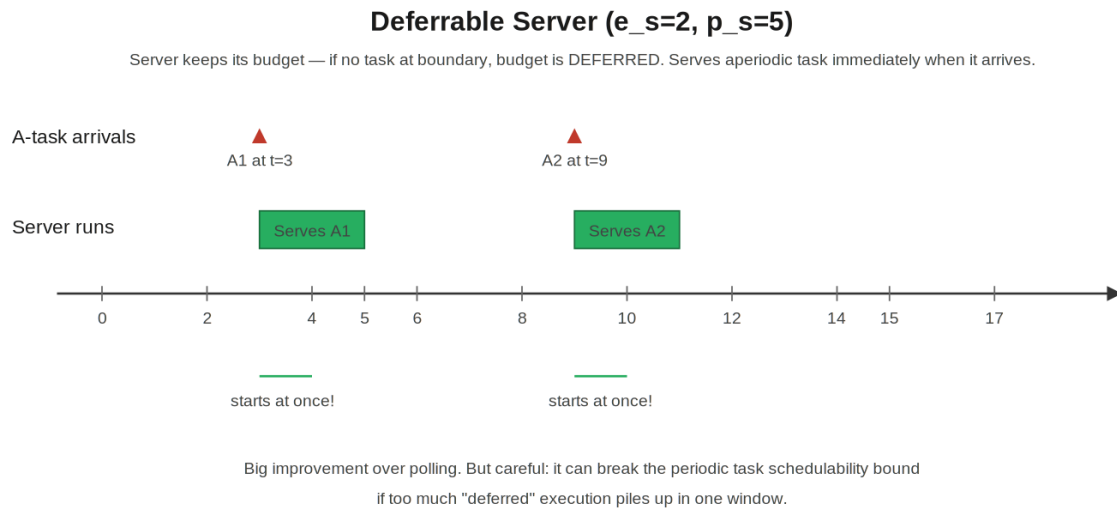


Figure 6. Deferrable server: aperiodic tasks A1 and A2 are served right away when they arrive.

Trade-off

Pro: much better response time for aperiodic tasks. Con: weaker schedulability guarantee for periodic tasks — the "deferred" execution can pile up in one period and push periodic tasks too far back. So the standard Liu-Layland bound has to be reduced.

6.3 Priority Exchange Server

Behaviour: When the server has budget but no aperiodic task to serve, instead of just keeping the budget idle, it SWAPS its high priority with a lower-priority periodic task. The lower-priority task gets to run "at" the server's high priority for the remaining budget. The server's priority temporarily drops.

- When an aperiodic task finally arrives, the server has to run at the **lower (exchanged)** priority — so response time is worse than the deferrable server.
- But: periodic tasks get more usable CPU time, so **periodic schedulability is better preserved**.

Where it sits

A compromise: between Polling (best for periodic, worst for aperiodic) and Deferrable (best for aperiodic, worst for periodic).

6.4 Sporadic Server (used in POSIX!)

Behaviour: Replenishes its budget ONE FULL PERIOD AFTER the moment a task starts using it (not at fixed period boundaries).

Why this is clever: it solves the "deferred build-up" problem of the deferrable server. By delaying replenishment, the server never causes more interference than a normal periodic task would. So it doesn't degrade the periodic utilization bound at all.

Real-world impact

The Sporadic Server is part of the POSIX real-time standard (SCHED_SPORADIC). Find it in real RTOSes like LynxOS and QNX.

6.5 Slack Stealer (Dynamic)

Idea: Don't create a separate server task at all. Instead, run a passive "stealer" that watches the schedule and detects unused slack — moments when no periodic task needs the CPU. It "steals" that time to run aperiodic tasks.

- Periodic tasks: hard guarantee, no deadlines missed.
- Aperiodic tasks: soft guarantee, but typically the BEST response time of all schemes.
- Cost: more runtime overhead because the stealer must constantly compute available slack.

Part 7: Comparison Table

Comparing Aperiodic Server Types

Server Type	Response Time	Schedulability	Key Idea	Category
Polling Server	Worst	Best for periodic	If no aperiodic at boundary, skip slot	Static
Deferrable Server	Best	Reduced bound	Keeps budget until task arrives	Static
Priority Exchange	Medium	Good	Swap priority with a low-prio periodic task	Static
Sporadic Server	Good	Doesn't degrade	Replenishes budget one period after use	Static (POSIX)
Slack Stealer	Best (dynamic)	Hard guarantee	Steals unused slack from periodic tasks	Dynamic

Figure 7. Side-by-side comparison of all aperiodic server types.

Summary: What Should Remember

- **When deadline < period:** use DMA, not RMA.
- **Real systems have context switches:** add $2C$ to each execution time and re-check schedulability.
- **Long critical tasks:** use period transformation to split them virtually.
- **Tasks with dependencies:** either use the right-to-left table algorithm OR retrofit deadlines/arrivals for EDF/RMA.
- **Aperiodic / sporadic tasks:** never mix them naively with RMA. Use a server.
- **Choice of server:** Polling (simple, slow), Deferrable (fast response, weaker bound), Priority Exchange (compromise), Sporadic (POSIX, best static option), Slack Stealer (best response, dynamic, more overhead).

Glossary (Quick Reference)

Term	Meaning
RMA	Rate Monotonic Algorithm. Static-priority scheduling: shorter period = higher priority.
DMA	Deadline Monotonic Algorithm. Like RMA but uses deadline instead of period.
EDF	Earliest Deadline First. Dynamic-priority scheduling.
Periodic task	Arrives at fixed intervals.
Aperiodic task	Arrives at random times, no minimum gap.
Sporadic task	Arrives at random times BUT with a guaranteed minimum gap.
Period (p, T)	Time between consecutive arrivals of a periodic task.
Execution time (e, C)	Worst-case CPU time the task needs.
Deadline (d, D)	Time by which the task must finish, measured from its arrival.
Context switch (CS)	OS overhead when switching from one task to another.
Schedulability	Can ALL tasks meet ALL deadlines under the given algorithm?
Utilization	Fraction of CPU used: sum of (e_i / p_i) .
Liu-Layland bound	Sufficient utilization condition for RMA: $U \leq N(2^{1/N} - 1)$.
Server budget	Maximum CPU time the server can use per period.
Replenishment	Refilling the server's budget.
Slack	Unused CPU time in the schedule.
Preemption	Stopping a running task to let a higher-priority one run.
Precedence constraint	Rule: Task B must wait for Task A to finish.
Transient overload	Short period when total demand temporarily exceeds CPU capacity.